

Evaluating Governed LLM-Driven ML Experimentation: Lifecycle, Provenance, and Failure Metrics

Anonymous Author(s)

Abstract

We propose a governance framework and evaluation methodology for LLM-driven autonomous ML experimentation, providing metrics and tooling for assessing agent reliability beyond task performance. Autonomous ML experimentation agents can propose code edits, execute training runs, and select follow-up trials, but task metrics alone do not reveal whether those actions were bounded, auditable, failure-aware, or reproducible. We argue that governed autonomous experimentation requires a complementary evaluation layer: a suite of *governance metrics*—acceptance rate, invalid-action rate, repeated-bad rate, provenance completeness, artifact capture completeness, and a failure taxonomy—that characterises the reliability of the experimentation process, not just its output. We present a governed harness that operationalises this evaluation framework. In the Agent = Model + Har-

ness decomposition [6], our harness owns trial lifecycle, scope enforcement, and audit; the LLM manager is a replaceable component. The framework separates proposal, execution, validation, and decision authority; enforces editable-scope constraints via a node specification; records append-only trial ledgers with complete provenance; and exports the full governance metric suite per campaign. We validate the framework on the ResNet-trigger ML benchmark with a five-trial real-worker campaign (complete provenance for all trials, documented failure taxonomy, 100% artifact completeness) and a memory ablation across two manager backends. The stochastic LangGraph ablation (10 trials per arm, `qwen2.5-coder:7b`) yields a non-monotonic result: the summary-only arm reduces repeated-bad rate from 0.78 to 0.44, while the rationale-augmented arm matches no-memory. This ordering holds in 2 of 3 independent replicates; a four-arm verbosity ablation finds that truncating rationale to 50 tokens does not restore summary-level performance, ruling out length as the causal mechanism. Task-metric AUC improves modestly (+0.000933) in the primary campaign as secondary evidence that the loop runs correctly; the central result is that lifecycle decisions, failures, and provenance are jointly auditable for every trial.

CCS Concepts

• **Computing methodologies** → **Artificial intelligence**;
• **Software and its engineering** → *Software testing and debugging*; • **General and reference** → *Evaluation*.

Keywords

agentic AI evaluation, autonomous ML experimentation, governed harness, control plane, append-only ledger, memory

ablation, failure taxonomy, reproducibility, bounded execution

ACM Reference Format:

Anonymous Author(s). 2026. Evaluating Governed LLM-Driven ML Experimentation: Lifecycle, Provenance, and Failure Metrics. In *Proceedings of KDD Workshop on Agentic AI Evaluation (KDD AAE'26)*. ACM, New York, NY, USA, 6 pages.

1 Introduction

Evaluating autonomous ML experimentation agents by final task score alone is insufficient. A higher validation AUC does not reveal whether the agent edited only allowed files, whether failed trials were preserved, whether no-op patches were rejected, whether keep/discard decisions were made by an external verifier, or whether the system repeatedly attempted known-bad ideas. Without explicit process controls, none of those properties are observable — and for scientific ML, observability of process determines whether an autonomous experiment is auditable.

This paper proposes a governance-metric evaluation protocol for autonomous ML experimentation and a harness that instantiates it. The evaluation question is not only *did the metric improve?* but *did the agent behave in a bounded, auditable, and reproducible way?* In the Agent = Model + Harness decomposition [6], the model proposes actions while the harness supplies execution boundaries, state tracking, and verification. We instantiate this as a governed control plane: a manager proposes one bounded change, a worker materialises the patch and run artifacts, and the control plane owns lifecycle, scope validation, metric parsing, keep/discard decisions, memory updates, and append-only records — none delegated back to the agent.

Concretely, the protocol reports six governance metrics alongside the task metric: acceptance rate, invalid-action rate, repeated-bad rate, provenance completeness, artifact capture completeness, and a failure taxonomy. These metrics expose behaviours that task score hides: invalid edit attempts, missing metrics, repeated degraded proposals, no-op patches, stale pending trials, and unauditable decisions. This framing targets the KDD Agentic AI Evaluation workshop's focus on measuring agent behaviour in real task settings.

We validate the protocol on two independent ML nodes. A five-trial real-worker campaign on the ResNet-trigger node demonstrates three-way lifecycle diversity, complete provenance, and a documented failure taxonomy. A stochastic LangGraph memory ablation shows that summary-only context partially reduces repeated-bad rate (0.78 to 0.44, 2 of 3 replicates); rationale-augmented memory is not reliably better, and a four-arm verbosity ablation rules out rationale

length as the causal mechanism. A second synthetic node (logistic regression, *LocalWorker*) confirms that the governance protocol operates identically on a different ML task without code changes.

Three contributions follow:

- governance metrics as first-class evaluation criteria for autonomous ML experimentation — acceptance, invalidation, repeated-bad, provenance completeness, artifact capture, and failure taxonomy;
- a governed control-plane protocol separating proposal, execution, validation, and keep/discard authority, with the manager as a replaceable component;
- an append-only audit schema recording proposal, patch, run log, parsed metric, decision, and failure category for every trial, validated across two nodes, three memory modes, and dedicated stress campaigns.

2 Background and Related Work

Autonomous experimentation sits at the intersection of coding agents, AutoML, experiment tracking, and harness engineering. The key distinction in this paper is that we evaluate the *harness* that governs the agent loop, not only the model that proposes code edits.

Autonomous code-space exploration. AIDE [3] is the closest academic system: both use LLMs to propose atomic code changes evaluated by a hard-coded metric. The structural difference is where lifecycle authority lives. AIDE couples search policy, evaluator, and summarisation inside one optimisation loop; this work separates proposal generation from governance. AIDE has no editable-path whitelist, no append-only audit ledger, and no failure taxonomy — it optimises the ML metric, whereas we govern the experimentation process. These are complementary goals; an AIDE-style search manager could be plugged into this harness, but it should not directly commit trial state.

Reproducibility and auditing. SHARP [1] uses an agent to reproduce prior ML experiments by re-executing documented procedures and surfacing discrepancies. SHARP governs reproduction of *existing* experiments; this paper governs autonomous *generation* of new ones. SHARP does not define trial lifecycle states, append-only ledgers, scope enforcement, or memory ablation protocols. Other systems — event-trace-based agents, deep-research-style accumulated learnings, and Hermes-style durable routines — inform implementation but do not provide an evaluation protocol for governed experimentation.

Harness engineering. The harness engineering literature independently validates this framing [2, 4, 6, 7]. Trivedy frames the harness as everything around the model: tools, sandboxes, memory, middleware, and execution hooks. Böckeler argues that trust in agent output requires constraining the solution space with computational guides and feedback sensors. OpenAI describes production agent work as an observability and feedback-loop problem: give the agent a map, not

Table 1: Workshop evaluation concerns mapped to harness mechanisms.

Evaluation concern	con-	Harness mechanism	Reported metric
Repeated poor behaviour		Append-only memory + repeated-bad detector	Repeated-bad rate
No ground truth for intermediate choices		Decision authority separated from manager; provenance IDs per trial	Provenance completeness
Deployment and scope safety		NodeSpec whitelist, no-op guard, pending-trial guard	Invalid-action rate
Long-horizon failure accumulation		State machine; terminal kept/disc./failed_inv.	Failure taxonomy
Benchmark reproducibility		NodeSpec YAML, deterministic patch bridge, seeded replicates	Artifact manifest

a manual, and turn failures into enforceable harness improvements. This paper instantiates those ideas: the node specification is a computational guide; the metric parser, pending-trial guard, and state machine are computational sensors; memory injection is an inferential guide; the repeated-bad detector is an inferential sensor.

Experiment tracking and AutoML. Trackers such as MLflow and Weights & Biases record outcomes after a run exists but do not govern the agent loop or classify invalid edit attempts before training. AutoML and neural architecture search systems optimise within predefined search spaces; they are not designed to audit self-directed exploration, explain why a proposed change was kept or failed, or expose lifecycle diversity. None of the above provides the full governed control plane: scope enforcement, append-only audit ledger, failure taxonomy, and memory ablation for evaluating autonomous experimentation.

3 Governed Control Plane

The system is a governed harness for autonomous ML experimentation. The manager proposes an experimental change; the worker materialises a patch and run artifacts; the control plane validates scope, executes the lifecycle, parses metrics, decides keep/discard/failed-invalid, updates memory, and appends the authoritative ledger record. Managers and workers are replaceable components; the control plane contract is fixed.

3.1 Alignment with Agent Evaluation Criteria

Table 1 maps the KDD Agentic AI Evaluation concerns to the harness mechanisms and metrics reported in this paper.

3.2 Manager, Worker, and Replacement Principle

The manager generates proposals (summary, rationale, target, intended effect). The worker executes bounded actions

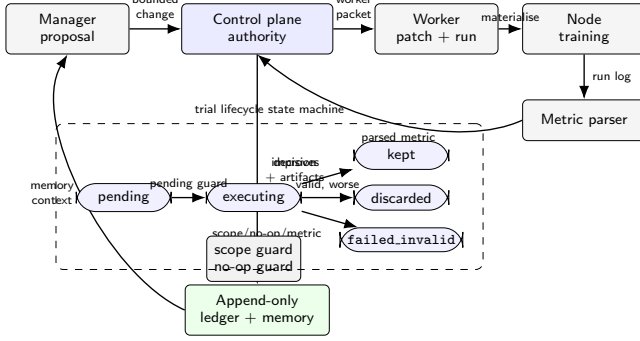


Figure 1: Governed trial lifecycle. The manager proposes, but the control plane owns guards, validation, decisions, append-only records, and memory updates. Terminal outcomes are kept, discarded, or failed_invalid.

(patch + run artifacts). The control plane owns authority: validity, state transitions, budget, memory updates, provenance IDs, and final decision. This separation is the *replacement principle*: any manager backend — baseline heuristic, `prompt_manager`, `LangGraphManager`, or an AIDE-style tree-search — can be swapped without changing the governance contract. Two worker implementations span the reproducibility spectrum: `LocalWorker` applies “Change PARAM from X to Y” directives with no external dependencies; `ClawWorker` invokes an external coding-agent path for free-form proposals, routing structured proposals through a deterministic constant-patch bridge that requires only Ollama.

3.3 Trial Lifecycle

Each trial follows the fixed lifecycle shown in Figure 1. Every opened budget slot must produce a `kept`, `discarded`, or `failed_invalid` audit object.

The control plane writes a pending guard before worker execution, making mid-trial crashes auditable. A scope validator checks the patch against the NodeSpec whitelist before training begins: patches touching frozen files are rejected, recorded, and prevented from corrupting node state. The append-only JSONL ledger stores, per trial: proposal summary and rationale, patch and log references, parsed metrics, validity status, decision, failure category, provenance IDs, and reproducibility hashes.

3.4 Memory Modes and Decision Authority

The memory interface has three modes: `none` (no prior-trial context), `append_only_summary` (trial summaries, decisions, metrics), and `append_only_summary_with_rationale` (summary augmented with per-trial failure rationales, optionally truncated via `rationale_max_tokens`). The ablation tests whether rationale-linked memory reduces repeated poor proposals more than summary alone.

The keep/discard decision lives in the control plane, not the manager. A valid candidate is kept only if the configured metric improves over the current best; a valid non-improving candidate is `discarded`; an invalid candidate is `failed_invalid` with a specific failure category. This deterministic, externally-owned decision rule addresses the systematic leniency in LLM self-evaluation [5].

4 Evaluation Protocol

The evaluation asks whether an autonomous ML experimentation loop is bounded, auditable, failure-aware, reproducible, and behaviorally affected by governance memory. Governance metrics are primary; task-metric AUC is secondary evidence that the loop runs end-to-end.

4.1 Benchmark Nodes

The primary benchmark is `resnet_trigger`, a real scientific ML task for near-threshold detector waveform binary classification. The editable surface is narrow: workers may modify only `nodes/ResNet_trigger/train.py`; all data, split logic, and node metadata are frozen. The task metric is validation AUC (`metric_direction=maximize`). A second benchmark, `lr_synthetic`, is a pure-NumPy logistic regression classifier (2000 samples, 10 features, 2 editable hyperparameters) run via `LocalWorker` for fully dependency-free protocol validation.

4.2 Campaign Protocol

Each campaign starts from a reset node state. The canonical five-trial campaign (`kdd_main_5trial`) uses `prompt_manager` with structured state-aware hyperparameter proposals and `claw_style_worker` with the deterministic patch path. The memory ablation runs equal budgets across three memory modes (`none`, `summary`, and `rationale-augmented summary`) under the same node, budget, parser, and reset discipline. Stress campaigns exercise forced failure paths at 100% rate. For every trial, the control plane records proposal, patch reference, raw log reference, parsed metrics, validity status, failure category, decision rationale, provenance IDs, and reproducibility hashes.

4.3 Memory Ablation Design

The pre-stated hypothesis is:

$$\text{RBR}(\text{none}) > \text{RBR}(\text{summary}) > \text{RBR}(\text{rationale})$$

where RBR is *repeated-bad rate*: the fraction of proposals repeating the same edit target and mechanism as a prior rejected trial without a new corrective rationale. Rationale-linked memory should reduce repeated-bad proposals more than raw summaries because it provides targeted failure signals rather than history noise [2, 4].

Two manager configurations are tested: a deterministic `prompt_manager` (design-constrained negative — round-robin makes memory structurally irrelevant) and `LangGraphManager` calling `qwen2.5-coder:7b` at temperature=0.7 with an explicit AVOIDANCE RULE. All LangGraph proposals are parsed into structured `param/old_value/new_value` fields and applied

Table 2: Governance metric suite. Given N trials, these metrics characterise process reliability independent of task-metric outcome.

Metric	Formal definition	What it detects
Acceptance rate	$ \text{kept} /N$	Valid improving edits
Invalid rate	$ \text{failed_invalid} /N$	Unevaluable trials
Repeated-bad rate	Repeated-bad proposals / N	Redundant failure modes
Provenance completeness	Trials with all five IDs / N	Independent reproducibility
Artifact capture	Trials with patch + log refs / N	Physical evidence retention
Failure taxonomy	Counts per FailureCategory	Classified, not hidden, failures

via the deterministic patch bridge, eliminating coding-agent dependencies. Three independent replicates are run; a four-arm verbosity ablation (`rationale_max_tokens=50`) tests whether rationale length drives the result.

4.4 Governance Metrics

Six failure categories are defined: scope violations, runtime errors, missing metrics, valid-but-worse edits (which produce **discarded** rather than **failed_invalid**), no-op patches, and agent edit failures. A campaign with high invalid rate but 100% provenance completeness is more interpretable than one with unexplained score changes and no audit trail.

5 Results

Results are presented in governance-first order. All numbers derive from ledgers in `experiments/ledgers/`; the full artifact suite is indexed in `artifact_manifest.json`.

5.1 Main Campaign Governance

Table 3 summarises the five-trial real-worker campaign on ResNet-trigger. Two valid improving edits are kept with complete patch diffs and provenance artifacts; three runtime-invalid trials are classified and recorded as first-class audit objects without corrupting node state. Provenance is complete for all five records (100%). The campaign does not include a **discarded** valid-but-worse trial; three-way lifecycle diversity is demonstrated across the evidence set (the 10-trial structured-manager extension produces seven real **discarded** trials with full provenance).

The net AUC gain (+0.000933) is smaller than baseline seed noise and should be treated as secondary evidence that the loop runs correctly, not as an optimisation claim.

5.2 Memory Ablation, Replication, and Verbosity Ablation

Table 4 reports the stochastic LangGraph ablation (qwen2.5-coder:7b, temperature=0.7, avoidance prompt, deterministic

Table 3: Main campaign governance (`kdd_main_5trial`, `ResNet-trigger`, `prompt_manager`, `claw_style_worker`, `budget=5`).

Field	Value
Kept / disc. / failed-invalid	2 / 0 / 3
Failure categories	$3 \times \text{runtime_error}$
Acceptance / invalid rate	0.40 / 0.60
Provenance completeness	1.00 (5/5)
Artifact capture	0.40 (patch for kept trials only, by design)
Initial / best val_auc	0.7738 / 0.7747 (+0.000933)
Baseline seed noise (5 seeds)	mean 0.785, 95% CI [0.775, 0.798]

patch bridge). All 30 trials across three arms use the deterministic patch path; no live coding agent is required.

Primary finding. The `append_only_summary` arm is the only configuration to consistently discover productive parameters (`TRAIN_FRACTION`, `LEARNING_RATE`), achieving best AUC = 0.8212 versus 0.7747 for both other arms. The non-monotonic ordering (`summary < none` $\approx$ rationale on RBR) holds in 2 of 3 replicates (rep1 and rep3; fails in rep2). It should be treated as suggestive rather than robust.

Verbosity ablation finding. Truncating rationale to 50 tokens does not restore summary-level performance: $\text{RBR}(C) = 0.778 = \text{RBR}(D) \neq \text{RBR}(B) = 0.444$. Rationale *length* is not the mechanism. Rationale *content* or avoidance-prompt format interactions remain as candidates.

Deterministic manager (design-constrained negative). A `prompt_manager` ablation produces identical outcomes across all three memory modes (2 kept, 3 failed-invalid per arm, $\text{RBR} = 0.40$) because the round-robin selector is insensitive to memory content. The memory subsystem functions correctly; the proposal generator is the constrained component.

5.3 Stress Campaigns and Second-Node Validation

Table 5 documents forced failure paths and the second-node governance validation.

The scope-violation and no-op cases are backed by real ledger records. The `1r_synthetic` campaign confirms that the governance protocol — scope enforcement, pending-guard lifecycle, three-way classification, complete provenance — operates identically on a different ML task and worker type without code changes, validating the NodeSpec pattern as a portable governance template.

6 Discussion and Limitations

6.1 Evidence Strength

Table 6 separates demonstrated properties from case-study findings and non-claims.

Table 4: LangGraph memory ablation summary (qwen2.5-coder:7b, temp=0.7, avoidance prompt, deterministic patch bridge). RBR = repeated-bad rate. Arm C tests whether rationale *length* drives arm D's failure.

Arm	Memory mode	RBR	Kept	Disc.	Best AUC	Observation
<i>Primary ablation — rep1, budget=10</i>						
A	none	0.78	1	9	0.7747	Baseline
B	summary	0.44	4	6	0.8212	Discovers <code>TRAIN_FRACTION</code>
D	full rationale	0.78	1	9	0.7747	Stuck on <code>BATCH_SIZE</code>
<i>Verbosity ablation — same node/budget/model</i>						
C	short rationale (50 tok)	0.78	1	9	0.7747	Matches D, not B: length is not the cause
<i>Seed replication — rep2, budget=10, independent seed</i>						
A	none	0.78	1	9	0.7747	
B	summary	0.67	2	8	0.7903	Ordering fails in this replicate
D	full rationale	0.56	3	7	0.8220	Rationale arm best in rep2
<i>Budget replication — rep3, budget=20</i>						
A	none	0.84	2	18	0.8031	
B	summary	0.79	3	17	0.8150	Ordering holds; finds <code>TRAIN_FRACTION</code> T5
D	full rationale	0.84	1	15+2	0.7747	Stuck on <code>TRACE_LEN</code>

Table 5: Stress campaigns and second-node validation. Each row is a silent failure in a naive unguarded loop that the harness makes visible.

Scenario	Campaign	Governed outcome	Evidence
Out-of-scope edit (frozen file touched)	<code>kdd_stress_scope</code>	Patch rejected before training; classified <code>invalid_edit_scope</code> ; git state unchanged	1/1 trials; patch ref retained
No-op patch (byte-identical edit)	<code>kdd_stress_noop</code>	Training skipped; classified <code>no_op_patch</code> ; empty patch hash recorded	1/1 trials; ledger record present
Second node: <code>lr_synthetic</code> (logistic regression, Local-Worker, no Ollama)	<code>lr_synth_baseline</code>	All three lifecycle outcomes in 5 trials; provenance complete; Node-Spec YAML portable	2 kept, 1 discarded, 2 failed-invalid; best <code>val_score</code> = 0.923

Table 6: Evidence strength by claim.

Claim	Evidence level	Where
Control plane classifies <code>kept</code> , <code>discarded</code> , <code>failed_invalid</code>	Demonstrated	Section 5.1 and stress campaigns
Append-only ledger preserves provenance	Demonstrated	100% completeness, Section 5.1
Governance metrics expose failures task score misses	Demonstrated	Stress campaigns, Section 5.3
Governance protocol is node-agnostic	Partial	<code>lr_synthetic</code> validation, Section 5.3
Summary memory reduces repeated-bad proposals	Case study (2/3 replicates)	Section 5.2
Rationale-augmented memory reduces repeated-bad rate	Not claimed	Non-monotonic; verbosity ablation also not supported

6.2 Limitations

Memory ablation: single node. Non-monotonic RBR ordering holds in 2 of 3 independent replicates and should be treated as suggestive. The verbosity ablation rules out rationale length but leaves content and avoidance-prompt format as open candidates. A stronger follow-up would ablate the avoidance prompt itself, use a wider editable surface, and report bootstrap confidence intervals.

Second-node scope. `lr_synthetic` validates protocol portability but uses a baseline manager. The memory ablation has not been repeated on the second node; whether the non-monotonic ordering generalises is unknown.

Discarded lifecycle in extended campaigns only. The primary five-trial campaign has no `discarded` trial. Three-way lifecycle diversity is demonstrated in the 10-trial extension and on `lr_synthetic` but not in the flagship run.

Worker dependencies. The `claw_style_worker` has two execution paths: deterministic patch (Ollama only) and free-form (requires a live AI coding agent). A pre-bridge LangGraph run produced 15/15 `edit_failed` outcomes when no agent was present — the control plane correctly refused all trials. The LangGraph ablation uses the deterministic path

exclusively. Fully dependency-free replication uses `LocalWorker` with only standard Python.

Task-metric noise. Five baseline seeds span AUC 0.771–0.811 (mean 0.785, 95% CI [0.775, 0.798]), larger than the primary campaign’s +0.000933 gain. AUC improvement should not be interpreted as evidence of meaningful optimisation.

Generalisation path. A stronger multi-node evaluation would add a holdout evaluation node (not used for development), repeat the memory ablation on each node with multiple seeds, compare manager backends under equivalent budgets, and report bootstrap confidence intervals for all governance metrics. The NodeSpec YAML pattern supports this without code changes; it is left to future work.

7 Conclusion

We presented an evaluation framework and governed harness for autonomous ML experimentation that treats trial lifecycle, validity, provenance, and failure handling as first-class evaluation objects. The central design choice is separation of authority: managers propose experiments, workers materialise patches and run artifacts, and the control plane owns scope enforcement, metric parsing, keep/discard decisions, memory updates, and append-only records.

On the ResNet-trigger node, the harness produces auditable real-worker campaigns with complete provenance, explicit failure categories, and reproducibility hashes. A memory ablation shows that summary-only context partially reduces repeated-bad proposals (0.78 to 0.44, 2 of 3 replicates); rationale-augmented memory is not reliably better, and truncating rationale to 50 tokens does not restore summary-level performance, suggesting that memory *format* rather than length determines avoidance effectiveness. The stronger conclusion is methodological: autonomous experimentation agents should be evaluated not only by task metrics, but by governance metrics that expose whether the experimentation process itself was bounded, reproducible, and scientifically inspectable.

References

- [1] Joschka Birk, Gregor Kasieczka, Siddharth Mishra-Sharma, Benjamin Nachman, Dennis Noll, and Tanvi Wamorkar. 2026. *A Scientific Human-Agent Reproduction Pipeline*. arXiv:2604.18752 [hep-ph] doi:10.48550/arXiv.2604.18752 SHARP: human-agent collaboration for reproducing scientific data analyses via a structured plan with human checkpoints. Demonstrated on ParticleNet-Lite jet classification..
- [2] Birgitta Böckeler. 2026. *Harness Engineering for Coding Agent Users*. Retrieved May 2026 from <https://www.thoughtworks.com/insights/blog/ai/harness-engineering-for-coding-agent-users> Thoughtworks Insights. Introduces the Guides (feedforward) × Sensors (feedback) × Computational/Inferential taxonomy instantiated in Table S1 of this paper..
- [3] Zhengyao Jiang et al. 2025. *AIDE: AI-Driven Exploration in the Space of Code*. arXiv:2502.13138 [cs.LG] doi:10.48550/arXiv.2502.13138 Weco AI. The most closely related academic system. Uses LLM-proposed atomic code changes evaluated by a hard-coded metric; couples search policy, evaluator, and summarisation in a single loop..
- [4] Alessandro Lopopolo et al. 2026. *Harness Engineering: Leveraging Codex in an Agent-First World*. Retrieved May 2026 from <https://openai.com/research/codex-harness-engineering> OpenAI Blog. Account of building a 1M-LOC product with zero manually-written code in 5 months. Key principle: give the agent a map, not

a manual; when the agent fails, identify what capability is missing and make it enforceable. Cited for the map-not-manual framing of rationale memory and the memory-ablation hypothesis..

- [5] Anu Rajasekaran et al. 2026. *Harness Design for Long-Running Application Development*. Retrieved May 2026 from <https://www.anthropic.com/engineering/harness-design-long-running> Anthropic Engineering Blog. Key finding: LLM self-evaluation is systematically lenient; separating generator from evaluator is required. Primary justification for keeping keep/discard authority in the control plane..
- [6] Vivek Trivedy. 2026. *The Anatomy of an Agent Harness*. Retrieved May 2026 from <https://blog.langchain.com/the-anatomy-of-an-agent-harness/> LangChain Blog. Canonical source for the Agent = Model + Harness decomposition used throughout this paper. Every component that is not the model is the harness: tools, sandboxes, memory, middleware, execution hooks..
- [7] Amanda Young et al. 2025. *Effective Harnesses for Long-Running Agents*. Retrieved May 2026 from <https://www.anthropic.com/engineering/effective-agents-long-running> Anthropic Engineering Blog. Identifies four agent failure modes and their structural fixes. Source for the failure-mode motivation table in Section 3..